

INTRODUCTION À

DAX

Data Analysis Expressions

Ludovic Moreau – moreau@artofnet.ch

Qu'est-ce que DAX?

Langage de programmation

Power BI

Analysis Services
Tabular

Power Pivot

Ressemble à Excel

Né avec
Power Pivot

Différences
majeures

Pas de
concept de
cellule

Points-clés

Contexte de
ligne

Contexte de
filtre

Time
intelligence

Agrégations

Créé pour les modèles de données

Données
relationnelles

Calculs de
contexte

Big data

LANGUAGE FUNCTIONNEL

Basé sur des fonctions
(comme Excel)
Pas 'Turing complete'

Le formatage est
important !

Ma mesure =

```
SUMX(FILTER(VALUES('Date'[Year]), 'Date'  
[Year]<2005), IF('Date'[Year]>=2000, [S  
ales Amount]*100, [Sales Amount]*90))
```

OUTIL DE FORMATAGE

Ma mesure = **SUMX** (

FILTER (

VALUES ('Date'[Year]),

'Date'[Year] < 2005

),

IF (

'Date'[Year] >= 2000,

[Sales Amount] * 100,

[Sales Amount] * 90

)

)

www.daxformatter.com

POURQUOI DAX EST-IL SI DIFFICILE ?

DAX: Tables, colonnes, expressions, mesures

Domaine

Outils

Définition / Requête / Analyse / Visuel

Mesures

Filtres

Implicite, explicite, volatile, rapide

Contexte de
ligne

Contexte de
filtre

Transition
de contexte

TYPE DE DONNEES ET GESTION

Numérique

Integer (64 bit)

Decimal (floating point)

Currency (money)

Date (DateTime)

TRUE / FALSE (Boolean)

Autres types

String

Binary Objects

Operator Overloading

Opérateurs non fortement typés, le résultat dépend des paramètres d'entrée.

Exemple:

"5" + "4" = 9, "5" + 4 = 9

5 & 4 = "54", "5" & 4 = "54"

La conversion se produit si nécessaire
(attention aux conversions non désirées)

10 REGLES POUR LES MEILLEURES PRATIQUES

- 1 Créer un modèle en étoile autant que possible
- 2 Commenter vos **expressions** abondamment.
- 3 Formater vos **expressions** avec daxformatter.com par exemple.
- 4 Utiliser des **variables** au lieu de répéter les mêmes calculs.
- 5 Créer des mesures à la place des colonnes.
- 6 Créer vos propres mesures explicites (plutôt qu'implicites).
- 7 Créer une table de référence de dates (+ temps si nécessaire).
- 8 Eviter les relations bidirectionnelles (utiliser **Dax** pour cela).
- 9 Utiliser une convention de nommage pour vos mesures, variables, colonnes, etc.
- 10 Créer une **table** séparée pour stocker vos mesures.

EXEMPLE PRATIQUE

A. Toujours formater votre code DAX.

B. Utilisez des noms en langage naturel (qui ont un sens).

C. Utilisez des noms de variables clairs précédé par «__».

D. Insérez un espace entre les différentes parenthèses fermantes.

E. Démarrez les noms des tables virtuelles par «__v» .

F. Toujours ajouter le nom de la table devant le nom d'une colonne.

G. Toujours utiliser [Nom Mesure] sans rien devant.

H. Insérez une ligne avant toute déclaration de variable.

I. Insérez des commentaires avec // ou /* */ (bloc).

J. Les noms des colonnes virtuelles devraient commencer par '@'.

Sales AVG 30Days =

```
VAR __vPeriod30D =  
    ADDCOLUMNS(  
        DATESINPERIOD ( 'Date'[Date],  
            MAX ( 'Date'[Date] ),  
            -30,  
            DAY  
        ),  
        "@Year Month", 'Date'[Year Month]  
    )  
VAR __FirstDayWithData =  
    CALCULATE (  
        MIN ( Sales[Order Date] ),  
        REMOVEFILTERS ()  
    )  
VAR __FirstDayInPeriod = // get first day  
    MINX (  
        __vPeriod30D,  
        'Date'[Date]  
    )  
VAR __Result = // compute result here  
    IF (  
        __FirstDayWithData <= __FirstDayInPeriod,  
        AVERAGEX (  
            __vPeriod30D,  
            [Sales Amount]  
        )  
    )  
RETURN __Result  
/* this measure computes the total sales  
For 3 months prior to current month */
```

FONCTIONS DAX

DAX.GUIDE

COMPTAGE

COUNT |
COUNTA
(compte tout sauf les blancs)
COUNTBLANK
(comptes les blancs et les vides)
COUNTROWS
(compte les lignes d'une table)
DISTINCTCOUNT
(compte les valeurs distinctes)

DATE & HEURE

DATE | **DATEVALUE** |
DAY | **EDATE** |
EOMONTH | **HOUR** |
MINUTE | **MONTH** |
NOW | **SECOND** |
TIME | **TIMEVALUE** |
TODAY | **WEEKDAY** |
WEEKNUM | **YEAR** |
YEARFRAC

FILTRE

ALL | **ALLEXCEPT** |
ALLSELECTED |
FILTER | **CALCULATE** |
CALCULATETABLE |
LOOKUPVALUE |
KEEPFILTERS |
ALLNOBLANKROW |
VALUES

LOGIQUE

TRUE | **AND** | **OR** |
NOT | **IF** | **IF.EAGER** |
SWITCH | **IFERROR**
Opérateurs:
AND (A, B) = A && B
OR (A, B) = A || B

TEXTE

FIND | **FIXED** |
FORMAT | **LEFT** | **LEN** |
LOWER | **MID** |
REPLACE | **REPT** |
RIGHT | **SEARCH** |
SUBSTITUTE | **TRIM** |
UPPER | **VALUE** |
EXACT |
CONCATENATE |
CONCATENATEX

FONCTIONS DAX

DAX.GUIDE

AGREGATIONS

Sur une colonne unique:

SUM | AVERAGE |
MIN | MAX | PRODUCT |
COUNT | COUNTA

ITERATEURS

SUMX | AVERAGEX |
MINX | MAXX | PRODUCTX |
COUNTX | COUNTAX |

INTELLIGENCE TEMP.

DATEADD | AVERAGE |
DATESMTD/QTD/YTD |
TOTALMTD/QTD/YTD |
DATESBETWEEN |
DATESINPERIOD |
PARALLELPERIOD |
SAMEPERIODLASTYEAR |
ENDOFMONTH/Q./YEAR |
NEXTDAY/MONTH/Q./YEAR
PREVIOUSDAY/MONTH/Q./
YEAR

MANIPULATION DE TABLE

ADDCOLUMNS | CROSSJOIN |
DATATABLE | ROW | UNION |
GENERATE |
GENERATESERIES |
SELECTCOLUMNS |
SUMMARIZE |
SUMMARIZECOLUMNS |
TOPN | TREATAS |
DISTINCT | VALUES |

INFORMATION

ISBLANK | ISEMPTY |
ISERROR | ISINSCOPE |
ISLOGICAL | ISNUMBER |
ISBOOLEAN | ISTEXT |
ISSTRING | ISDATETIME |
SELECTEDMEASURE |

MATH & ARRONDI

FLOOR | TRUNC |

ROUNDDOWN |

MROUND | ROUND |

CEILING | ROUNDUP |

ABS | CONVERT | EXP |

POWER | FACT | MOD |

SIGN | PI | LN | LOG |

SQRT | DIVIDE

RELATIONS

RELATED

Suis une relation et retourne la valeur dans la colonne liée.

Sales[PriceVariance] = Sales[Unit Price] * **RELATED**(Product[Unit Price])

RELATEDTABLE

Suis une relation et retourne toutes les lignes liées à la ligne courante. La longueur de la chaîne de relations n'importe pas mais elles doivent toutes suivre la même direction.

Customer[NumOFBuys] = **COUNTROWS**(**RELATEDTABLE**(Sales))

MOVINGAVERAGE

RUNNINGSUM

FIRST

LAST

COLLAPSE

EXPAND

NEXT

PREVIOUS

Les fonctions VISUAL DAX ne sont pas très nombreuses à ce jour mais permettent de créer des analyses complexes sans taper une ligne de code. La restriction est qu'elles ne s'appliquent qu'aux données visibles dans le visuel.

<pre> 1 Moving SUM CM + LM = 2 SUMX(RANGE(-1), [Sales Amount]) // RANGE retrieves a table containing the (-1) row here and the current </pre>						
Calendar Month	Calendar Date	Sales Amount	Running sum	Running sum LOWESTPARENT	Running sum HIGHESTPARENT	Moving SUM CM + LM
January 2007	02.01.2007	48'646 CHF	48'646.02	48'646.02	48'646.02	48'646.02
	03.01.2007	92'244 CHF	140'890.09	140'890.09	140'890.09	140'890.09
	04.01.2007	13'950 CHF	154'840.38	154'840.38	154'840.38	106'194.36
	05.01.2007	62'051 CHF	216'891.21	216'891.21	216'891.21	76'001.12
	07.01.2007	23'306 CHF	240'196.84	240'196.84	240'196.84	85'356.46
	09.01.2007	20'543 CHF	260'740.19	260'740.19	260'740.19	43'848.98
	10.01.2007	6'566 CHF	267'305.75	267'305.75	267'305.75	27'108.91
	11.01.2007	22'693 CHF	289'998.80	289'998.80	289'998.80	29'258.61
	12.01.2007	16'252 CHF	306'250.44	306'250.44	306'250.44	38'944.68
	13.01.2007	11'315 CHF	317'565.49	317'565.49	317'565.49	27'566.69
	15.01.2007	58'225 CHF	375'790.36	375'790.36	375'790.36	69'539.92
	16.01.2007	45'596 CHF	421'386.01	421'386.01	421'386.01	103'820.52
	17.01.2007	29'601 CHF	450'986.56	450'986.56	450'986.56	75'196.20
	18.01.2007	91'281 CHF	542'267.06	542'267.06	542'267.06	120'881.05
	19.01.2007	27'932 CHF	570'199.48	570'199.48	570'199.48	119'212.92

Opérateur **IN**

Vérifie si le résultat d'une expression est présent dans une liste de valeurs:

```
FILTER( Customers,  
Customer[State] IN { "WA", "NY", "CA" }  
)
```

Filtre la table Customers en ne gardant que les lignes où la colonne [State] vaut "WA" ou "NY" OU "CA" (**IN** remplace des fonctions **OR** multiples).

Opérateur **AND** (2 arguments. au max)

```
AND([country]=«CH», [Quantity] > 0)
```

```
[country]=«CH» && [Quantity] > 0 && [Year] = 2020
```

Opérateur **OR** (2 arguments. au max)

```
OR([country]=«CH», [Qty] > 0)
```

```
[country]=«FR» || [country]=«CH» || [Qty] > 0
```

Concaténation de chaînes

```
VAR mytext = "hello" & " " & "world"
```

OPERATEURS - CONSTRUCTEUR

Constructeur de table { }

// table à 1 colonne

{ "Red" , "Green" , "Blue" }

	[Value]
1	Red
2	Green
3	Blue

/* une table de plusieurs colonnes, chaque ligne est incluse entre parenthèses. */

{ ("Red" , "Green" , "Blue") }

	[Value1]	[Value2]	[Value3]
1	Red	Green	Blue

Applications

Product[Color] **IN** { "Red" , "Green" , "Blue" }

(Dates[Year], Dates[Monthnum])=(2017,12)

(Dates[Year], Dates[Monthnum]) **IN** { (2017,12) , (2018,1) }

```
{  
  ("A", 10, 2.5 , DATE(2025,12,24), TRUE() ),  
  ("B", 20, 5 , DATE(2025,12,25), TRUE() ),  
  ("C", 30, 7.5 , DATE(2025,12,26), FALSE() )  
}
```

	[Value1]	[Value2]	[Value3]	[Value4]	[Value5]
1	A	10	2.5	12/24/2025 12:00:00 AM	True
2	B	20	5	12/25/2025 12:00:00 AM	True
3	C	30	7.5	12/26/2025 12:00:00 AM	False

AGREGATEURS - ITERATEURS

AGREGATEURS

SUM, MIN, MAX, AVERAGE, MEDIAN, STDEV, etc.

Applicables sur une seule colonne d'une table.

AVGdelivery :=

```
AVERAGE(  
    Sales[DaysToDeliver]  
)
```

ITERATEURS

SUMX, MINX, MAXX, AVERAGEX, MEDIANX, STDEVX, etc.

En cas d'agrégation d'une expression, il faut utiliser un itérateur:

AVGDelivery :=

```
AVERAGEX(  
    Sales,  
    INT(Sales[Deliverydate] -  
        Sales[Orderdate])  
)
```

LA FONCTION SWITCH (2 UTILISATIONS)

DAX.GUIDE

// Applique des fonctions SI multiples

SizeDesc =

SWITCH (

Product[Size],

"S", "Small",

"M", "Medium",

"L", "Large",

"XL", "Extra Large",

"Other"

)

// Equivalent à la fonction SI.conditions

DiscountPct =

SWITCH (

TRUE (),

// 1^{er} test

Product[Size] = "S", 0.5,

// 2^e test

AND (Product[Size] = "L", Product[Price] < 100), 0.2,

// 3^e test

Product[Size] = "L", 0.35,

// valeur par défaut

0

)

FONCTION DIVIDE

DAX.GUIDE

/* DIVIDE permet d'éviter les tests IF lors d'une division en vérifiant les zéros dans les dénominateurs.

C'est aussi plus rapide que IF.*/

```
IF (
    Sales[SalesAmount] <> 0,
    Sales[GrossMargin] / Sales[SalesAmount],
    0
)
```

/* Il vaut mieux utiliser DIVIDE*/

```
DIVIDE (
    Sales[GrossMargin],
    Sales[SalesAmount],
    0
)
```

VARIABLES

Très pratique pour réutiliser un calcul déjà effectué. Nécessaire pour éviter de subir une modification du context de filtre. Utilisation recommandée dès 10 lignes de code.

```
VAR TotalQuantity = SUM ( Sales[Quantity] )
RETURN
IF (
    TotalQuantity > 1000,
    TotalQuantity * 0.95,
    TotalQuantity * 1.25
)
```

/ TotalQuantity est évaluée une fois lors du RETURN et devient ensuite une 'constante'.*

```
AverageSalesPerCustomer :=
AVERAGEX(Customer, [Sales Amount])
```

```
CustomersbuyingMoreThanAvarage :=
VAR AVGsales = [AverageSalesPerCustomer]
```

```
RETURN
```

```
COUNTROWS(
    FILTER(Customer,
        [Sales Amount] > AVGsales)
)
```

/ Nous ne souhaitons pas que la moyenne soit calculée dans FILTER. AVGsales est évaluée dans le contexte avant d'être utilisée comme constante.*/*

VARIABLES DE TYPE TABLE

/* Une variable peut contenir un scalaire ou une table.

L'utilisation de variables de type table permet de décomposer des expressions complexes. */

```
VAR __vSalesGreaterThan10 = FILTER ( Sales, Sales[Quantity] > 10 )
```

```
RETURN
```

```
SUMX (
```

```
    FILTER (
```

```
        __vSalesGreaterThan10,
```

```
        RELATED ( Product[Color] ) = "Red"
```

```
    ),
```

```
    Sales[Amount]
```

```
)
```

LA FONCTION FILTER

DAX.GUIDE

// FILTER est un itérateur qui parcourt la table Sales et filtre les lignes où l'expression est vraie.

```
VAR __vSalesGreaterThan10 = FILTER ( Sales, Sales[Quantity] > 10 )
```

// __vSalesGreaterThan10 contient toutes les lignes de Sales où qty > 10

```
RETURN
```

```
SUMX (
    FILTER (
        __vSalesGreaterThan10,
        RELATED ( Product[Color] ) = "Red"
        /* SUMX somme la colonne [Amount] de la table __vSalesGreaterThan10 en ne
        gardant que les lignes où la colonne [Color] est égale à «red» */
    ),
    Sales[Amount]
)
```


VALUES et DISTINCT retourne une table d'une colonne contenant la liste des valeurs uniques de son argument.

```
VALUES( Product[Color])
```

```
// contient toutes les valeurs uniques de la colonne [Color]
```

```
DISTINCT( Product[Color])
```

```
// Renvoie le même résultat dans ce cas.
```

DISTINCT ne tient pas compte d'une éventuelle valeur BLANK(), notamment en cas de relation manquante.

CONTEXTE D'ÉVALUATION

Contexte de filtre

C'est la somme de tous les filtres s'appliquant au calcul d'une mesure.

Ces filtres peuvent provenir :

- d'une valeur en ligne ou en colonne dans un visuel.
- d'un segment.
- d'un filtre appliqué à un visuel / page / rapport.
- d'une expression de filtre dans une fonction **CALCULATE**.

Contexte de ligne

Apparaît lors du calcul d'une colonne calculée ou quand on évalue une expression dans une itération.

Le contexte de ligne permet de parcourir les lignes successives d'une table et de connaître les valeurs des autres colonnes sur cette ligne.

Le contexte de filtre filtre le modèle de données, le contexte de ligne parcourt les lignes d'une table.

LA FONCTION ALL()

DAX.GUIDE

ALL(tablename)

Renvoie toutes les lignes d'une table y compris les duplicatas.

ALL(table[columnName])

Retourne la liste des valeurs uniques de la table.

ALL(table[col1], table[col2])

Retourne toutes les combinaisons uniques des colonnes indiquées.

ALL()

Sans paramètres, **ALL()** ne peut être utilisée que comme modificateur de la fonction **CALCULATE()**. Elle supprime alors tous les filtres du contexte de filtre.

ALL() EN TANT QUE FONCTION DE TABLE

EVALUATE

```
CALCULATETABLE (  
    ALL ( 'Product'[Color] ),  
    'Product'[Color] = "Red"  
)  
ORDER BY 'Product'[Color]
```

'Product'[Color] = "Red" is placed in the filter context.

CALCULATETABLE then selects all the distinct values of the [color] column, ignoring the filter context.

Color
Azure
Black
Blue
Brown
Gold
Green
Grey
Orange
Pink
Purple
Red
Silver
Silver Grey
Transparent
White
Yellow

EVALUATE

```
CALCULATETABLE (  
    ALL (  
        'Product'[Brand],  
        'Product'[Color]  
    ),  
    'Product'[Color] = "Red"  
)  
ORDER BY  
    'Product'[Brand],  
    'Product'[Color]
```

'Product'[Color] = "Red" is placed in the filter context.

CALCULATETABLE then selects all the unique combinations of values in the [color] and [brand] columns, ignoring the filter context.

Brand	Color
A. Datum	Azure
A. Datum	Black
...	...
Contoso	Black
Contoso	Blue
...	...
The Phone Company	Black
The Phone Company	Gold
...	...
Wide World Importers	Black
Wide World Importers	Gold
...	...
Wide World Importers	White
Wide World Importers	Yellow

ALL() EN TANT QUE MODIFICATEUR DE CALCULATE

EVALUATE

CALCULATETABLE (

```
{  
    ( "Sales Amount ", [Sales Amount] ),  
  
    ( "Sales Amount (ALL Colors)",  
      CALCULATE ( [Sales Amount], ALL ( 'Product'[Color] ) ) ),  
  
    ( "Sales Amount (ALL Products)",  
      CALCULATE ( [Sales Amount], ALL ( 'Product' ) ) ),  
  
    ( "Sales Amount (ALL)",  
      CALCULATE ( [Sales Amount], ALL ( ) ) ),  
  
    ( "Sales Amount (ALL Sales)",  
      CALCULATE ( [Sales Amount], ALL ( Sales ) ) )  
},  
'Product'[Color] = "Red",  
'Product'[Category] = "audio",  
'Date'[Calendar Year] = "CY 2008" )
```

Value1	Value2
Sales Amount	20 224,17
Sales Amount (ALL Colors)	105 363,42
Sales Amount (ALL Products)	9 927 582,99
Sales Amount (ALL)	30 591 343,98
Sales Amount (ALL Sales)	30 591 343,98

ALLEXCEPT() EN TANT QUE MODIFICATEUR

ALLEXCEPT(tableName, columnName)

Utilisé avec **CALCULATE** ou **CALCULATETABLE**, **ALLEXCEPT** retire les filtres de la table étendue spécifiée en 1er argument, ne conservant que le(s) filtre(s) spécifié(s) dans les arguments suivants.

DEFINE

MEASURE Sales[# Sales] = **COUNTROWS** (Sales)

EVALUATE

CALCULATETABLE (

```
{
  ( 1, "# Sales (CY 2009 - Red)", [# Sales] ),
  ( 2, "# Sales (CY 2009)", CALCULATE ( [# Sales], ALLEXCEPT ( Sales, 'Date' ) ) ),
  ( 3, "# Sales (Red)", CALCULATE ( [# Sales], ALLEXCEPT ( Sales, 'Product'[Color] ) ),
  ( 4, "# Sales", CALCULATE ( [# Sales], REMOVEFILTERS ( ) ) )
}
```

'Product'[Color] = "Red",

'Date'[Calendar Year] = "CY 2009"

)

Value1	Value2	Value3
1	# Sales (CY 2009 – Red)	2,038
2	# Sales (CY 2009)	39,793
3	# Sales (Red)	5,802
4	# Sales	100,231

LA FONCTION ALLEXCEPT()

DAX.GUIDE

ALLEXCEPT(tableName, columnName)

En tant que fonction de table, **ALLEXCEPT** évalue toutes les combinaisons des colonnes dans la table qui ne sont pas indiquées dans le 2e argument de la fonction. Dans ce cas, le résultat garde uniquement les colonnes de la table, PAS de la table étendue (utilisation peu commune).

EVALUATE

CALCULATETABLE (

```
{  
    ( "CALCULATE FILTER", CALCULATE (  
        COUNTROWS ( Sales ),  
        ALLEXCEPT ( Sales, 'Date' )),  
    ( "TABLE FUNCTION", COUNTROWS ( ALLEXCEPT ( Sales, 'Date' )),  
},  
'Date'[Calendar Year] = "CY 2009",  
'Product'[Color] = "Red"  
)
```

Value1	Value2
CALCULATE FILTER	39,793
TABLE FUNCTION	100,231

LE MODIFICATEUR ALLSELECTED()

Renvoie toutes les lignes d'une table ou toutes les valeurs d'une colonne, en ignorant les filtres qui ont pu être appliqués à l'intérieur de la requête, mais en conservant les filtres provenant de l'extérieur. Elle est particulièrement utile lorsqu'une mesure doit calculer un pourcentage par rapport au total des catégories dans les lignes d'une matrice.

Pct =

Brand	Brand	Sales Amount	Pct
☐ A. Datum	Adventure Works	2,761,057.66	36.52%
☒ Adventure Works	Contoso	2,227,244.32	29.46%
☒ Contoso	Fabrikam	990,275.08	13.10%
☒ Fabrikam	Litware	506,104.50	6.69%
☒ Litware	Northwind Traders	119,857.67	1.59%
☒ Northwind Traders	Proseware	956,335.76	12.65%
☒ Proseware	Total	7,560,874.99	100.00%
☐ Southridge Video			
☐ Tailspin Toys			
☐ The Phone Company			
☐ Wide World Importers			

DIVIDE (

[Sales Amount],

CALCULATE (

[Sales Amount],

ALLSELECTED ('Product'[Brand])

)

)

LA FONCTION ALLSELECTED()

ALLSELECTED peut aussi être utilisées en tant que fonction mais ceci n'est pas recommandé.

EVALUATE

UNION (

CALCULATETABLE (

{ **COUNTROWS (ALLSELECTED (Product)) },**

Product[Category] = "Audio"

) ,

{ **COUNTROWS (ALLSELECTED (Product)) }**

)

2 rows

Value
115
2517

TOPN(N_VALUE, TABLE)

[DAX.GUIDE](#)

-- TOPN might return more than the requested rows in presence of ties.

EVALUATE

```
TOPN (
    3,
    ADDCOLUMNS (
        VALUES ( 'Product'[Product Name] ),
        "@Sales Amount",
        MROUND ( [Sales Amount], 500000 )
    ),
    [@Sales Amount],
    DESC
)
ORDER BY [@Sales Amount] DESC
```

Product Name	@Sales Amount
Adventure Works 26" 720p LCD HDTV M140 Silver	1500000
Contoso Projector 1080p X980 White	500000
SV 16xDVD M360 Black	500000
A. Datum SLR Camera X137 Grey	500000
Contoso Telephoto Conversion Lens X400 Silver	500000

-- Multiple sorting criteria can be provided in further parameters.

EVALUATE

```
TOPN (
    3,
    ADDCOLUMNS (
        VALUES ( 'Product'[Product Name] ),
        "@Sales Amount",
        MROUND ( [Sales Amount], 500000 )
    ),
    [@Sales Amount],
    DESC,
    [Product Name],
    ASC
)
ORDER BY [@Sales Amount] DESC
```

Product Name	@Sales Amount
Adventure Works 26" 720p LCD HDTV M140 Silver	1500000
Contoso Projector 1080p X980 White	500000
A. Datum SLR Camera X137 Grey	500000

RANKX(TABLE, EXPRESSION)

Attention à la transition de contexte lorsqu'on utilise RANKX.

RANK1 = **RANKX**(
 VALUES('Product'[Brand]),
 [Sales Amount])

RANK2 = **RANKX**(
 ALLSELECTED('Product'[Brand]),
 [Sales Amount])

Brand	Sales Amount	rounded to 1M	RANK1	RANK2
Contoso	7'352'399 CHF	7'000'000 CHF	1	1
Fabrikam	5'554'016 CHF	6'000'000 CHF	1	2
Adventure Works	4'011'112 CHF	4'000'000 CHF	1	3
Litware	3'255'704 CHF	3'000'000 CHF	1	4
Proseware	2'546'144 CHF	3'000'000 CHF	1	5
A. Datum	2'096'185 CHF	2'000'000 CHF	1	6
Wide World Importers	1'901'957 CHF	2'000'000 CHF	1	7
Northwind Traders	1'040'552 CHF	1'000'000 CHF	1	10
Southridge Video	1'384'414 CHF	1'000'000 CHF	1	8
The Phone Company	1'123'819 CHF	1'000'000 CHF	1	9
Tailspin Toys	325'042 CHF	0 CHF	1	11
Total	30'591'344 CHF	31'000'000 CHF	1	1

RANKX(TABLE, EXPRESSION)

```
RANK SKIP = IF(  
    ISINSCOPE('Product'[Brand]),  
    RANKX(  
        ALLSELECTED('Product'[Brand]),  
        [rounded to 1M],  
        ,  
        DESC,  
        Skip    ))
```

Brand	Sales Amount	rounded to 1M	RANK1	RANK2	RANK SKIP	RANK DENSE
Contoso	7'352'399 CHF	7'000'000 CHF	1	1	1	1
Fabrikam	5'554'016 CHF	6'000'000 CHF	1	2	2	2
Adventure Works	4'011'112 CHF	4'000'000 CHF	1	3	3	3
Litware	3'255'704 CHF	3'000'000 CHF	1	4	4	4
Proseware	2'546'144 CHF	3'000'000 CHF	1	5	4	4
A. Datum	2'096'185 CHF	2'000'000 CHF	1	6	6	5
Wide World Importers	1'901'957 CHF	2'000'000 CHF	1	7	6	5
Northwind Traders	1'040'552 CHF	1'000'000 CHF	1	10	8	6
Southridge Video	1'384'414 CHF	1'000'000 CHF	1	8	8	6
The Phone Company	1'123'819 CHF	1'000'000 CHF	1	9	8	6
Tailspin Toys	325'042 CHF	0 CHF	1	11	11	7
Total	30'591'344 CHF	31'000'000 CHF	1	1		

Utiliser **ISINSCOPE** pour éviter de prendre en compte les totaux (p.ex) dans le calcul du rang.

Vous pouvez utiliser **ALL** (retire le contexte de filtre) ou **ALLSELECTED** (garde les filtres externes) sur la colonne.

Évitez **VALUES** à cause de la transition de contexte.

CALCULATE(EXPRESSION,[FILTER1],...)

```
/* calcule la mesure [Sales Amount] pour  
toutes les couleurs, quel que soit le  
contexte de filtre. */
```

```
SalesAllColors =
```

```
CALCULATE (  
    [Sales Amount],  
    ALL ( Product[Color] )  
)
```

```
/* Le contexte de filtre est appliqué au  
modèle de données. Les colonnes de la table  
Product filtrent aussi la table Sales  
(étendue). */
```

```
SalesRedProducts =
```

```
CALCULATE (  
    [Sales Amount],  
    Product[Color] = "Red"  
)
```

Il existe aussi **CALCULATETABLE** qui permet d'évaluer une table en modifiant le contexte de filtre.

CALCULATE AVEC KEEPFILTERS(), AJOUT DE FILTRES

```
// Calculer les ventes pour les produits rouges et bleus en gardant en compte les filtres externes.
```

```
SalesTrendyColors =
```

```
CALCULATE (
```

```
[Sales Amount],
```

```
KEEPFILTERS (
```

```
Product[Color] IN { "Red", "Blue" } )
```

```
)
```

```
// KEEPFILTERS garde les filtres originaux qui s'ajoutent aux filtres de CALCULATE.
```


CALCULATE: FILTRES MULTIPLES

// L'utilisation de plusieurs filtres dans CALCULATE générera une intersection de ces filtres dans le contexte.

CALCULATE (

...,

// Filter expression #1

Product[Color] **IN** { "Red", "Black" },

// Filter expression #2

FILTER (

ALL (Date[Year], Date[Month]),

OR (

AND (Date[Year] = 2006, Date[Month] = "December"),

AND (Date[Year] = 2007, Date[Month] = "January")

)))

CALCULATE: REMPLACER UN FILTRE

/* Plusieurs CALCULATE imbriqués ne créent pas une intersection de filtres mais une autre operation appelée OVERWRITE.

Dans l'exemple ci-dessous, le filtre (intérieur) Yellow/Black remplace le Black/Blue (extérieur).

Le résultat final ne prendra en compte que les couleurs jaune et noir. */

```
CALCULATE (  
    CALCULATE (  
        ...,  
        Product[Color] IN { "Yellow", "Black" }  
    ),  
    Product[Color] { "Black", "Blue" }  
)
```

CALCULATE IMBRIQUES: INTERSECTION DE FILTRES

```
//KEEPFILTERS garde les filtres originaux au lieu de les remplacer.
```

```
CALCULATE (  
    CALCULATE (  
        ...,  
        KEEPFILTERS ( Product[Color] IN { "Yellow", "Black" } )  
    ),  
    Product[Color] IN { "Black", "Blue" }  
)
```

Au final, seul 'black' reste visible dans le contexte de filtre. En effet, 'Black' est la seule valeur issue de l'intersection des deux conditions de filtre provenant des deux fonctions CALCULATE imbriquées.

CALCULATE : RESUME

DAX manipule les filtres de la manière suivante:

- **AJOUT** (filtres multiples dans **CALCULATE**)
- **REEMPLACEMENT** (**CALCULATE** imbriqués)
- **EFFACEMENT** (grâce à **ALL** ou **REMOVEFILTERS**)
- **INTERSECTION** (en utilisant **KEEPFILTERS**)

TRANSITION DE CONTEXTE

Lors de l'utilisation d'une mesure ou de CALCULATE dans une expression calculée pendant une itération, une transition de contexte sera automatiquement appliquée. Les champs de la ligne seront ajoutés au contexte de filtre.

EVALUATE

ADDCOLUMNS (

VALUES ('Date'[Calendar Year]),
"QtySum", SUM(Sales[Quantity])

Calendar Year	QtySum
CY 2005	140 180
CY 2006	140 180
CY 2007	140 180
CY 2008	140 180
CY 2009	140 180
CY 2010	140 180
CY 2011	140 180

EVALUATE

ADDCOLUMNS (

VALUES ('Date'[Calendar Year]),
"QtyCalc", CALCULATE (SUM(Sales[Quantity]))

Calendar Year	QtyCalc
CY 2005	(Blank)
CY 2006	(Blank)
CY 2007	44 310
CY 2008	40 226
CY 2009	55 644
CY 2010	(Blank)
CY 2011	(Blank)

ETAPES D' EVALUATION DANS CALCULATE

Etapes



DEFINE

MEASURE Sa

SUMX (

VALUES (

CALCULATE

[Sales

'Produ

KEEPF

USERE

EVALUATE

SUMMARIZECC

'Product'[Col

"Sales Amour

)

DEFINE

1. Evaluation des arguments de filtre dans le contexte de filtre original.
2. Si un contexte de ligne est présent, CALCULATE opère la transition de context sinon, l'étape est passée.
3. Evaluation des modificateurs de CALCULATE
USERELATIONSHIP , CROSSFILTER , ALL ,
ALLEXCEPT, ALLSELECTED , ALLNOBLANKROW
4. Application des filtres explicites de CALCULATE et de
KEEPFILTERS.

Color	Sales Amount
Silver	99 923,48
Blue	99 923,48
White	99 923,48
Red	99 923,48
Black	99 923,48
Green	99 923,48
Orange	99 923,48
Pink	99 923,48
Yellow	99 923,48
Purple	99 923,48
Brown	99 923,48
Grey	99 923,48
Gold	99 923,48
Azure	99 923,48
Silver Grey	99 923,48
Transparent	99 923,48

MODIFICATEUR CROSSFILTER

NumofColors :=

```
CALCULATE (
    DISTINCTCOUNT (Product[Color] ),
    CROSSFILTER (Sales[ProductKey] , 'Product'[ProductKey] , BOTH )
)
```

CROSSFILTER modifie la relation dans le modèle de données entre les deux colonnes passées en arguments pendant le calcul de la mesure (mais ne modifie pas le modèle de façon permanente).

Les opérateurs BOTH (relation bidirectionnelle), NONE (aucune relation) et ONESIDE (relation unidirectionnelle) sont disponibles dans cette fonction.

MODIFICATEUR USERELATIONSHIP

Delivered Amount :=

```
CALCULATE (
    [Sales Amount],
    Dates[Year] = «CY 2007»,
    USERELATIONSHIP (Dates[Date] , 'Sales[DeliveryDate] )
)
```

USERELATIONSHIP active la relation mentionnée pendant le calcul de la mesure (cette relation doit avoir été créée au préalable et exister en tant que relation inactive dans le modèle de données).

CALCULATE VS FILTER

```
VAR __c1 = SUMX ( FILTER ( ALL ( Sales ), RELATED ( 'Product'[Color] ) = "red" ),  
                  Sales[Quantity] )
```

```
VAR __c11 = CALCULATE (  
    SUMX ( ALL( Sales ), Sales[Quantity] ),  
    FILTER ( ALL ( Sales ), RELATED ( 'Product'[Color] ) = "red" ) )
```

```
VAR __c2 = CALCULATE (  
    SUMX ( ALL (Sales ), Sales[Quantity] ),  
    'Product'[Color] = "red" )
```

```
VAR __c3 = CALCULATE ( SUMX ( Sales, Sales[Quantity] ),  
    ALL ( Sales ),  
    'Product'[Color] = "red" )
```

```
RETURN
```

```
ROW ( "c1", __c1, "c11", __c11, "c2", __c2, "c3", __c3 )
```

c1	c11	c2	c3
8 079	140 180	140 180	8 079

CALCULATE –COLONNES MULTIPLES DANS UN FILTRE

DEFINE

```
MEASURE Sales[Big Sales Amount] =
```

```
    CALCULATE (
```

```
        [Sales Amount],
```

```
        KEEPFILTERS (Sales[Quantity] * Sales[Net Price] > 1000 ) )
```

```
// KEEPFILTERS necessary not to override the qty in the rows of the matrix
```

EVALUATE

```
SUMMARIZECOLUMNS (
```

```
    Sales[Quantity],
```

```
    "Sales Amount", [Sales Amount],
```

```
    "Big Sales Amount", [Big Sales Amount]
```

```
)
```

Quantity	Sales Amount	Big Sales Amount
1	17,569,935.65	4,097,102.62
2	2,951,553.65	1,516,230.38
4	5,735,255.24	4,424,075.64
3	4,334,599.44	2,865,005.14

CALCULATE – FILTRE SUR COLONNES DE PLUSIEURS TABLES

Qty red >1 = CALCULATE (

[Sales Amount],

Sales[Quantity] > 1 || 'Product'[Color] = "Red"

)



Cette mesure ne fonctionne pas car la condition de filtre utilise des colonnes de 2 tables différentes.

SOLUTION: Utilisez CROSSJOIN, GENERATE, SUMMARIZE ou SUMMARIZECOLUMNS pour générer les combinaisons de valeurs permettant de filtrer la table des ventes.

CALCULATE (

[Sales Amount],

FILTER (

SUMMARIZE (

Sales,

'Product'[Color],

Sales[Quantity]

),

'Product' [Color] = "Red" || Sales[Quantity] > 1

)

)



CALCULATE – FILTRE SUR UNE MESURE

Sales10K =

```
CALCULATE( [Sales Amount],  
            [Sales Amount] > 10000 )
```

La mesure ne peut pas être utilisée dans
l'argument de filtre.

SOLUTION: Utiliser FILTER pour générer l'expression
de filtre utilisant une mesure.

```
CALCULATE( [Sales Amount],  
            FILTER(Sales,  
                    [Sales Amount] > 1000  
            )  
)
```

RESTRICTIONS & RECOMMENDATIONS

Il existe des restrictions qui s'appliquent aux expressions booléennes lorsqu'elles sont utilisées comme arguments de filtre de CALCULATE:

- Impossible de référencer des colonnes à partir de plusieurs tables.
- Impossible de référencer une mesure.
- Impossible d'utiliser les fonctions de CALCULATE imbriquées.
- Impossible d'utiliser des fonctions qui analysent ou retournent une table.

Il est recommandé de passer des arguments de filtre en tant qu'expressions booléennes, dans la mesure du possible car ils sont explicitement optimisés pour filtrer efficacement les colonnes de cette façon.

Attention aux transitions de contexte lors de l'utilisation de CALCULATE ou d'une mesure dans un itérateur.



DAX